

Python Language Programming

Types, Operators, Variables

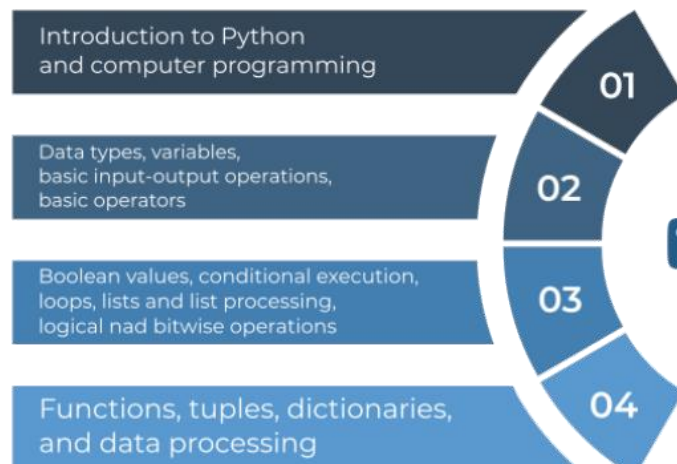
- **Disclaimer**

- These slides and notes are based on the contents of the courses:

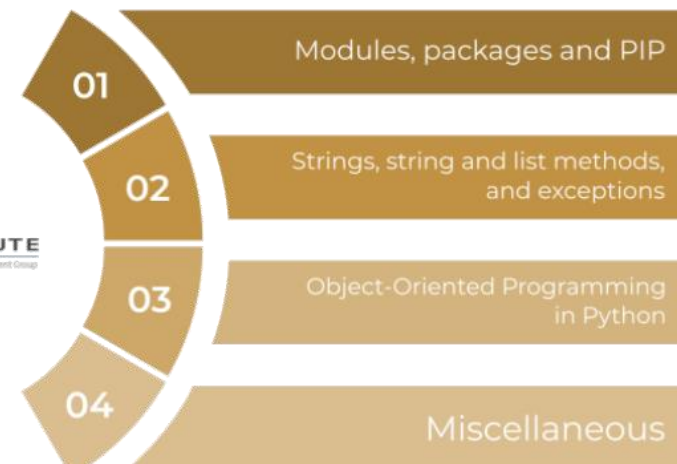
- **Python Essentials 1 and 2**

- aligned with the **PCEP** and **PCAP** certifications, respectively;
- developed by the OpenEDG Python Institute;
(www.pythoninstitute.org)
- offered in the Cisco Networking Academy program.

Python Essentials 1



Python Essentials 2



- **Disclaimer**

- These slides and notes are based on the contents of the courses:

- **Python Essentials 1 and 2**

- aligned with the **PCEP** and **PCAP** certifications, respectively;
- developed by the OpenEDG Python Institute;
(www.pythoninstitute.org)
- offered in the Cisco Networking Academy program.

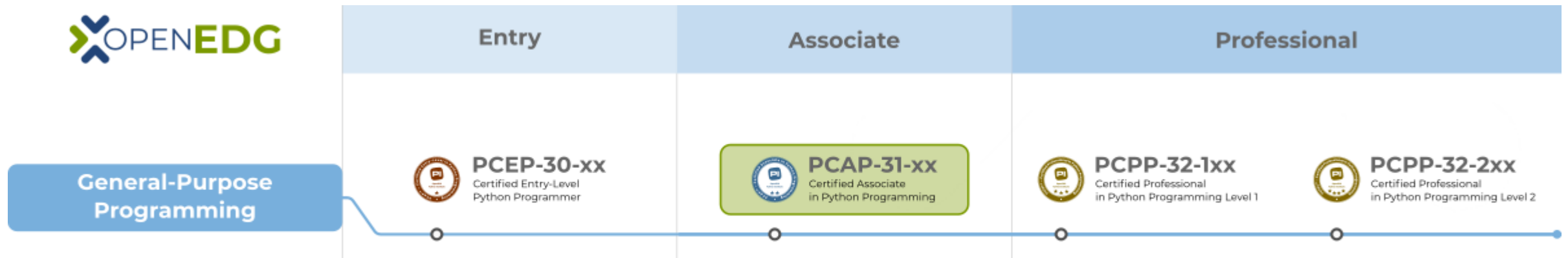


Table of Contents

- **Types**

- Integer

- Float

- **String**

- Functions

- Indexing

- Slicing

- Escape Sequences

- Methods

- Operators (+, *)

- **Boolean**

- **Casting**

- `int()`, `float()`, `str()`

- **Operators**

- Arithmetic

- Logical

- Precedence

- **Variables**

- Rules for naming a variable

- Shortcut Operators

- **Bibliography**

Python Language Programming

- **Types**

- Integer
- Float
- String
- Boolean
- List
- Tuple
- Set
- Dict

Type	Example
int	1, 2, 3
float	3.14, 2.5, 6.
string	"Hello World!"
bool	True, False uppercase "T" and "F"
list	[1, 2, 3, 4], ["aaa", "bbb", "ccc"]
tuple	(1, 2, 3, 4), ("aaa", "bbb", "ccc")
set	{1, 2, 3, 4}, {"aaa", "bbb", "ccc"}
dict	{key1: value1, key2:value2}

Python Language Programming

- **Types**

- **Integer**

- those which are devoid of the fractional part

```
>>> print(0o123) #octal representation
83
>>> print(0x123) #hexadecimal representation
291
>>> print(123)   #decimal representation
123
```

- those with a fractional part

```
>>> type(var1)
<class 'float'>
>>> type(var2)
<class 'float'>
>>> type(var3)
<class 'int'>
>>> type(var4)
<class 'float'>
```

Python always chooses the more economical form of the number's presentation

Python Language Programming

- **Types**

- **String**

- an immutable sequence of characters
 - CANNOT be used:
 - **del** instruction, **append** method and **insert** method

```
alphabet = "bcdefghijklmnopqrstuvwxy"  
  
alphabet = "a" + alphabet  
alphabet = alphabet + "z"  
  
print(alphabet)
```



abcdefghijklmnopqrstuvwxyz

Python Language Programming

- **Types**

- **String**

- there are two ways to define strings in Python:
 - **one-line strings:** cannot cross line boundaries – denoted using either apostrophes ('string') or quotes ("string")
 - **multi-line strings:** can occupy more than one line of source code, delimited by trigraphs ('''string''') or ("""string""")

Note: the multi-line strings include the line feed character (\n)

```
oneline1 = 'Line1'
oneline2 = "Line2 - double quotes"

print(len(oneline1))
print(len(oneline2))

multiline1 = '''Line #3
Line #4'''
multiline2 = '''Line #5
Line #6 - double quotes'''

print(len(multiline1))
print(len(multiline2))
```



5
21
15
32

Python Language Programming

- **Types**

- **String**

- **Functions**

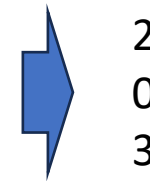
- **len() function**

- returns the number of characters contained by the arguments

```
# Example 1  
word = 'by'  
print(len(word))
```

```
# Example 2  
empty = ''  
print(len(empty))
```

```
# Example 3  
i_am = 'I\'m'  
print(len(i_am))
```



2
0
3



Python Language Programming

- Types
 - String
 - Functions
 - `ord()`
 - returns the character's ASCII/UNICODE code point value
 - `chr()`
 - takes a code point and returns its character.

Dec	Hex	Name	Char	Ctrl-char	Dec	Hex	Char	Dec	Hex	Char	Dec	Hex	Char
0	0	Null	NUL	CTRL-@	32	20	Space	64	40	@	96	60	`
1	1	Start of heading	SOH	CTRL-A	33	21	!	65	41	A	97	61	a
2	2	Start of text	STX	CTRL-B	34	22	"	66	42	B	98	62	b
3	3	End of text	ETX	CTRL-C	35	23	#	67	43	C	99	63	c
4	4	End of xmit	EOT	CTRL-D	36	24	\$	68	44	D	100	64	d
5	5	Enquiry	ENQ	CTRL-E	37	25	%	69	45	E	101	65	e
6	6	Acknowledge	ACK	CTRL-F	38	26	&	70	46	F	102	66	f
7	7	Bell	BEL	CTRL-G	39	27	'	71	47	G	103	67	g
8	8	Backspace	BS	CTRL-H	40	28	(	72	48	H	104	68	h
9	9	Horizontal tab	HT	CTRL-I	41	29	)	73	49	I	105	69	i
10	0A	Line feed	LF	CTRL-J	42	2A	*	74	4A	J	106	6A	j
11	0B	Vertical tab	VT	CTRL-K	43	2B	+	75	4B	K	107	6B	k

```
# Demonstrating the ord() function.
```

```
char_1 = 'a'
```

```
char_2 = ' ' # space
```

```
print(ord(char_1))
```

```
print(ord(char_2))
```

```
# Demonstrating the chr() function.
```

```
print(chr(97))
```

```
print(chr(100))
```

97
32
a
d

Python Language Programming

- **Types**

- **String**

- **Functions**

- **max()** – finds the character with the maximal codepoint;
 - **min()** – finds the character with the minimal codepoint.
 - **sorted()** - creates a new, sorted list;

```
# Demonstrating the sorted() function:  
first_greek = ['omega', 'alpha', 'pi', 'gamma']  
first_greek_2 = sorted(first_greek)  
print(first_greek)  
print(first_greek_2)
```



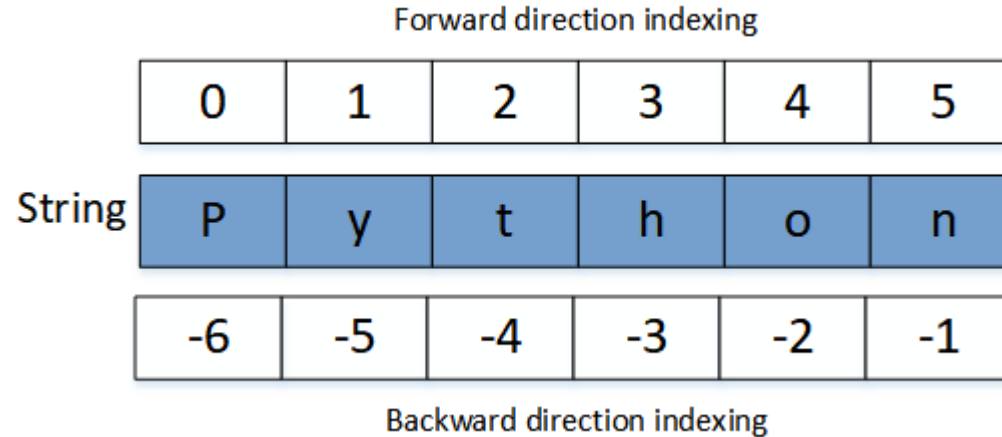
```
['omega', 'alpha', 'pi', 'gamma']  
['alpha', 'gamma', 'omega', 'pi']
```

Python Language Programming

- Types

- String

- Indexing



```
# Indexing strings.  
the_string = 'silly walks'  
for ix in range(len(the_string)):  
    print(the_string[ix], end=' ')  
print()
```


silly walks

```
# Iterating through a string.  
the_string = 'silly walks'  
for character in the_string:  
    print(character, end=' ')  
print()
```


silly walks

- **Types**

- **String**

- **Slicing**

- creation of a new sub-string from a given string on the basis starting and ending indexes:
 - **arr[start:stop]** # from start to **stop-1**
 - **arr[start:]** # from start to the rest of the array
 - **arr[:stop]** # from the beginning to **stop-1**
 - **arr[:]** # copy the entire array
 - **arr[start:stop:step]** # from start to **stop-1** using step

```
alpha = "abcdefg"

print(alpha[1:3])
print(alpha[3:])
print(alpha[:3])
print(alpha[3:-2])
print(alpha[-3:4])
print(alpha[:2])
print(alpha[1::2])
```



bc
defg
abc
de

aceg
bdf

Python Language Programming

- **Types**

- **String**

- **in and not in operators**

- checks if its left argument (a string) can be found anywhere within the right argument (another string).

```
alphabet = "abcdefghijklmnopqrstuvwxyz"

print("f" in alphabet)
print("F" in alphabet)
print("1" in alphabet)
print("ghi" not in alphabet)
print("Xyz" not in alphabet)
```



True
False
False
False
True

Python Language Programming

- Types
 - String
 - Escape sequences

Escape Sequence	Meaning
<code>\n</code>	New Line
<code>\t</code>	Horizontal Tab
<code>\b</code>	BackSpace
<code>\r</code>	Carriage Return
<code>\a</code>	Audible bell
<code>\'</code>	Printing single quotation
<code>\"</code>	printing double quotation
<code>\?</code>	Question Mark Sequence
<code>\\</code>	Back Slash
<code>\f</code>	Form Feed
<code>\v</code>	Vertical Tab
<code>\0</code>	Null Value
<code>\nnn</code>	Print octal value
<code>\xhh</code>	Print Hexadecimal value

Python Language Programming

- **Types**

- **String**

- **Methods**

- `find()` - returns the index of first occurrence of a substring
 - `sort()` - sorts the list, containing strings, in situ
 - `capitalize()` – changes all string letters to capitals;
 - `center()` – centers the string inside the field of a known length;
 - `count()` – counts the occurrences of a given character;
 - `join()` – joins all items of a tuple/list into one string;
 - `lower()` – converts all the string's letters into lower-case letters;
 - `lstrip()` – removes the white characters from beginning of the string;
 - `replace()` – replaces a given substring with another;

Python Language Programming

- **Types**

- **String**

- **Methods**

- `rfind()` – finds a substring starting from the end of the string;
 - `rstrip()` – removes the trailing white spaces from the end of string;
 - `split()` – splits the string into a substring using a given delimiter;
 - `strip()` – removes the leading and trailing white spaces;
 - `swapcase()` – swaps the letters' cases (lower to upper and vice versa)
 - `title()` – makes the first letter in each word upper-case;
 - `upper()` – converts all the string's letter into upper-case letters.

Python Language Programming

- Types

- String

- Operators

- Concatenation

- the **+** (plus) sign, when applied to two strings, becomes a concatenation operator

```
fnam = input("May I have your first name, please? ")
lnam = input("May I have your last name, please? ")
print("Thank you.")
print("\nYour name is " + fnam + " " + lnam + ".")
```



```
May I have your first name, please? Luis
May I have your last name, please? Veloso
Thank you.
```

```
Your name is Luis Veloso.
```

Python Language Programming

- Types

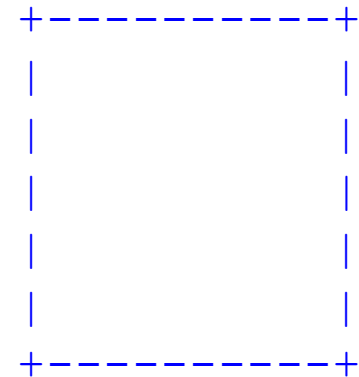
- String

- Operators

- Replication

- the ***** (**asterisk**) sign, when applied to a string and number becomes a replication operator:

```
print ("+" + 10 * "-" + "+")  
print ( ("|" + " " * 10 + "| \n") * 5, end="")  
print ("+" + 10 * "-" + "+")
```



Python Language Programming

- **Types**

- **Boolean**

- boolean can have True or False values
 - almost any value is evaluated to True if it has some sort of content:
 - any string is True, except empty strings;
 - any number is True, **except 0**;
 - any list, tuple, set, and dictionary are True, except empty ones.
 - the value None is evaluated as False

Python Language Programming

- Types

- Casting

- **int()**: takes one argument (e.g., a string: `int(string)`) and tries to convert it into an integer;

```
value = int(5.5)
print('Reciprocal of', value, 'is', 1/value)
```



Reciprocal of 5 is 0.2

- **float()**: takes one argument (e.g., a string: `float(string)`) and tries to convert it into a float.

```
value = int(float("5.5"))
print('Reciprocal of', value, 'is', 1/value)
```



Reciprocal of 5 is 0.2

- **str()**: takes one argument (e.g., a string: `str(integer)`) and tries to convert it into a string.

```
value = str(5.5)
print('Reciprocal of', value, 'is', 1/5.5)
```



Reciprocal of 5.5 is
0.18181818181818182

Python Language Programming

- Types
- Casting

```
value = int(5.5)
print('Reciprocal of', value, 'is', 1/value)

value = int(float("5.5"))
print('Reciprocal of', value, 'is', 1/value)

value = int("5.5")
print('Reciprocal of', value, 'is', 1/value)
```



```
Reciprocal of 5 is 0.2
Reciprocal of 5 is 0.2
Traceback (most recent call last):
  File "main.py", line 7, in
    <module>
      value = int("5.5")
ValueError: invalid literal for
int() with base 10: '5.5'
```

Python Language Programming

- Types
 - Casting

```
In [1]: float(5)
```

```
Out[1]: 5.0
```

```
In [2]: int(5.3)
```

```
Out[2]: 5
```

```
In [3]: int("5")
```

```
Out[3]: 5
```

```
In [4]: int("Y")
```

```
-----  
ValueError                                Traceback (most recent call last)  
<ipython-input-4-c37fe004ffa0> in <module>  
----> 1 int("Y")
```

```
ValueError: invalid literal for int() with base 10: 'Y'
```

```
In [5]: str(1)
```

```
Out[5]: '1'
```

```
In [6]: str(4.32)
```

```
Out[6]: '4.32'
```


Python Language Programming

- Operators
 - Arithmetic

Operator	Operator name	Example	Returns
+(Unary)	Unary positive	+1 or +a	The positive value (not necessary)
-(Unary)	Unary negative	-1 or -a	The negative value
+	Addition	1 + 2 or 'a + b	The value when a is added to b - the sum of numbers
-	Subtraction	2 - 1 or b - a	The value when a is subtracted from b
*	Multiplication	1 * 2 or a * b	The product of a and b
/	Division	2 / 1 or b / a	The quotient of diving b by a. The result is always a float
%	Modulo	2 % 1 or b % a	The remainder of dividing b by a
//	Floor division	2 // 1 or b // a	The quotient of b divided by a, rounded to the smallest whole number.
**	Exponentiation	2 ** 1 or b ** a	The value of b raised to the power a

Python Language Programming

- Operators
 - Arithmetic

Python operation	Arithmetic operator	Algebraic expression	Python expression
Addition	+	$f + 7$	<code>f + 7</code>
Subtraction	-	$p - c$	<code>p - c</code>
Multiplication	*	$b \cdot m$	<code>b * m</code>
Exponentiation	**	x^y	<code>x ** y</code>
True division	/	x/y or $\frac{x}{y}$ or $x \div y$	<code>x / y</code>
Floor division	//	$\lfloor x/y \rfloor$ or $\left\lfloor \frac{x}{y} \right\rfloor$ or $\lfloor x \div y \rfloor$	<code>x // y</code>
Remainder (modulo)	%	$r \bmod s$	<code>r % s</code>

Python Language Programming

- Operators
 - Arithmetic

```
print(2 ** 3)
print(2 ** 3.)
print(2. ** 3)
print(2. ** 3.)
```

```
8
8.0
8.0
8.0
```

```
print(2 * 3)
print(2 * 3.)
print(2. * 3)
print(2. * 3.)
```

```
6
6.0
6.0
6.0
```

```
print(7 / 4)
print(7 / 4.)
print(7. / 4)
print(7. / 4.)
```

```
1.75
1.75
1.75
1.75
```

```
print(7 // 4)
print(7 // 4.)
print(7. // 4)
print(7. // 4.)
```

```
1
1.0
1.0
1.0
```

* The exponentiation operator uses right-sided binding, e.g., $2 ** 2 ** 3 = 256$

Python Language Programming

- Operators
 - Logical

Operator	C	Python
AND	&&	and
OR		or
NOT	!	not
Equality	==, !=	==, !=, is, is not
Comparasion	<, >, <=, >=	<, >, <=, >=

==, != verify if two objects have the **same value**

```
a = [1, 2, 3]
b = [1, 2, 3]
a == b          #True
```

is, is not verify if two objects are the **same object**

```
a = [1, 2, 3]
b = [1, 2, 3]
a is b          #False
```

Python Language Programming

- Operators

- Logical

- equality operator (==)

- binary operator with left-sided binding. It needs two arguments and checks if they are equal.

- =

- is an assignment operator
 - e.g., `a = b` assigns `a` with the value of `b`;

- ==

- question: are these values equal?
 - `a == b` compares `a` and `b`.

Python Language Programming

- Operators
 - Logical

Operator	Meaning
<code>==</code>	Equal to
<code>!=</code>	Not equal to
<code>></code>	Greater than
<code><</code>	Less than
<code>>=</code>	Greater than or equal to
<code><=</code>	Less than or equal to

Python Language Programming

- Operators
- Precedence

Precedence	Operator Sign	Operator Name
Highest	**	Exponentiation
	+X, -X, ~X	Unary positive, unary negative, bitwise negation
	*, /, //, %	Multiplication, division, floor, division, modulus
	+, -	Addition, subtraction
	<<, >>	Left-shift, right-shift
	&	Bitwise AND
	^	Bitwise XOR
		Bitwise OR
	==, !=, <, <=, >, >=, is, is not	Comparison, identity
	not	Boolean NOT
	and	Boolean AND
Lowest	or	Boolean OR

Python Language Programming

- **Variables**

- A variable comes into existence as a result of assigning a value to it.

```
var = 1
account_balance = 1000.0
client_name = 'John Doe'
print(var, account_balance, client_name)
print(var)
```



```
1 1000.0 John Doe
1
```

```
a = 3.0
b = 4.0
c = (a ** 2 + b ** 2) ** 0.5
print("c =", c)
```



```
c = 5.0
```

$$c = \sqrt{a^2 + b^2}$$

Python Language Programming

- **Variables**

- **Rules for naming a variable:**

- must be composed of:
 - upper-case or lower-case letters;
 - digits;
 - character `_` (underscore).
 - must begin with a letter or underscore:
 - NOT a digit!
 - must not be any of Python's reserved words:
 - 'False', 'None', 'True', 'and', 'as', 'assert', 'break', 'class', 'continue', 'def', 'del', 'elif', 'else', 'except', 'finally', 'for', 'from', 'global', 'if', 'import', 'in', 'is', 'lambda', 'nonlocal', 'not', 'or', 'pass', 'raise', 'return', 'try', 'while', 'with', 'yield'

Python Language Programming

- Variables
- Shortcut Operators

Operator	Example	Equivalent Expression (m=15)	Result
=	y = <u>a+b</u>	y = 10 + 20	30
+=	m +=10	m = m+10	25
-=	m -=10	m = m-10	5
*=	m *=10	m = m*10	150
/=	m /=10	m = m/10	1.5
%=	m %=10	m = m%10	5
=	m **=2	m = m2 or $m = m^2$	225
//=	m //=10	m = m//10	1

Bibliography

- Burkov, A. (2019). The hundred-page machine learning book. Andriy Burkov.
- **Cisco. (2022). Python Essentials. Retrieved September 4, 2024, from <https://www.netacad.com/>**
- Geron, A. (2019). Hands-on machine learning with scikit-learn, keras, and tensorflow. O'Reilly.
- Ekman, M. (2022). Learning Deep Learning. Addison-Wesley.
- Mckinney, W. (2017). Python for data analysis: Data wrangling with pandas, numpy, and ipython. O'Reilly.
- Raschka, S., & Mirjalili, V. (2019). Python machine learning: Machine learning and deep learning with python, scikit-learn, and tensorflow. Packt Publishing.
- Russel, S., & Norvig, P. (2018). Artificial intelligence: A modern approach. Pearson Education Limited.